

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДВФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ

Департамент математики и компьютерного моделирования

Отчет
к лабораторной работе №9 по дисциплине «Суперкомпьютеры и параллельное программирование»

По теме: «Задачи продолжения»

Вариант 20

Выполнил: студент группы Б9122-01.03.02мкт

_____ Чирич И.С.
подпись *Ф.И.О.*

Проверил: Доцент

_____ Селезнев Т.Э.
подпись *Ф.И.О*

г. Владивосток
2025

Содержание

Постановка задачи **3**

Индивидуальное задание **4**
Написание программного кода 4
Тестирование 8

Вопросы для защиты работы **9**

Выводы **11**

Постановка задачи

Цель данной лабораторной работы: изучить механизм работы задач продолжения.

Задачи лабораторной работы:

- Научиться формирвоать последовательность запуска задач
- Научиться использовать задачи продолжения для формирования последовательности выполняемых работ
- Научиться использовать задачи продолжения для реализации массового запуска задач

Индивидуальное задание

Написание программного кода

Создадим шаблон консольного проекта при помощи утилиты `dotnet`:

```
dotnet new console -n continue_tasks
```

В корне проекта создастся файл Project.cs, в котором и будет написан код лабораторной работы.

Создаем класс Program с методом Main:

```
class Program
{
        static void Main(string[] args)
        {

        }
}
```

В соответствии с индивидуальным заданием, опишем три статических метода, которые будут запускаться как задачи продолжения.

Функция `SumEvenDiv4`, которая получает матрицу и считает сумму её элементов, делящихся нацело на 4:

```
static void SumEvenDiv4(Task task, object o)
{
        Console.WriteLine("Задача {0} начала выполнение...", Task.CurrentId);
        List<List<int>> matrix = (List<List<int>>)o;
        int sum = 0;
        foreach (List<int> list in matrix)
        {
                foreach (int i in list)
                {
                        if (i % 2 == 0 && i % 4 == 0) sum += i;
                }
        }
        Thread.Sleep(2000);
        Console.WriteLine("Задача {0} завершена успешно. Сумма элементов: {1}", Task.
            CurrentId, sum);
}
```

Функция `FindMaxDiv3`, которая получает матрицу и находит максимальный элемент, который делится на 3:

```
static void FindMaxDiv3(Task task, object o)
{
        Console.WriteLine("Задача {0} начала выполнение...", Task.CurrentId);
        List<List<int>> matrix = (List<List<int>>)o;
        int maximum = int.MinValue;
        foreach (List<int> list in matrix)
```

```
        {
                foreach (int i in list)
                {
                        if (i % 3 == 0 && maximum < i) maximum = i;
                }
        }
        Thread.Sleep(1000);
        Console.WriteLine("Задача {0} завершена успешно. Максимальное значение: {1}",
           Task.CurrentId, maximum);
}
```

Функция FindMin, которая получает матрицу и находит её минимальный элемент:

```
static void FindMin(Task task, object o)
{
        Console.WriteLine("Задача {0} начала выполнение...", Task.CurrentId);
        List<List<int> > matrix = (List<List<int>>)o;
        int minimum = int.MaxValue;
        foreach (List<int> list in matrix)
        {
                foreach (int i in list)
                {
                        if (i < minimum) minimum = i;
                }
        }
        Thread.Sleep(1500);
        Console.WriteLine("Задача {0} завершена успешно. Минимальное значение: {1}",
           Task.CurrentId, minimum);
}
```

Заметим, что параметрами передаются объекты типа Task. Это важно, так как это дает возможность получать результат работы предыдущей задачи.

Реализуем метод Main. Его задача - сгенерировать матрицу по заданным параметрам и запустить цепочку Task-ов:

```
static void Main(string[] args)
{
        Random random = new Random();
        const int MAX_VALUE = 1_000_000;
        const int MAX_SIZE = 10_000;
        int MATRIX_SIZE = random.Next(MAX_SIZE);
        List<List<int>> matrix = new List<List<int>>();
        Task task_base = new Task(() => {
                Console.WriteLine("Основная задача {0} начала выполнение...", Task.
                   CurrentId);
                for (int i = 0; i < MATRIX_SIZE; i++)
                {
                        matrix.Add(new List<int>());
                        for (int j = 0; j < MATRIX_SIZE; j++)
                        {
```

```
                                matrix[i].Add(random.Next(MAX_VALUE));
                        }
                }
                Console.WriteLine("Основная задача {0} завершила выполнение.\Созданан
                    матрица размера {1} на {1}", Task.CurrentId, MATRIX_SIZE);

        });

        Task task_continue_1 = task_base.ContinueWith(SumEvenDiv4, matrix);
        Task task_continue_2 = task_base.ContinueWith(FindMaxDiv3, matrix);
        Task task_continue_3 = task_base.ContinueWith(FindMin, matrix);

        task_base.Start();
        Console.ReadKey();
}
```

Заметим, что в методе `Main` определяется базовая задача через лямбда-функцию, в которой происходит создание матрицы.

Итоговый код в листинге ниже:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.InteropServices;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace continue_tasks
{
        internal class Program
        {
                static void Main(string[] args)
                {
                        Random random = new Random();
                        const int MAX_VALUE = 1_000_000;
                        const int MAX_SIZE = 10_000;
                        int MATRIX_SIZE = random.Next(MAX_SIZE);
                        List<List<int>> matrix = new List<List<int>>();
                        Task task_base = new Task(() => {
                                Console.WriteLine("Основная задача {0} начала
                                    выполнение...", Task.CurrentId);
                                for (int i = 0; i < MATRIX_SIZE; i++)
                                {
                                        matrix.Add(new List<int>());
                                        for (int j = 0; j < MATRIX_SIZE; j++)
                                        {
                                                matrix[i].Add(random.Next(MAX_VALUE));
                                        }
                                }
                                Console.WriteLine("Основная задача {0} завершила
```

```
                                выполнение.\Созданап матрица размера {1} на {1}",
                                Task.CurrentId, MATRIX_SIZE);
31
32                      });
33
34                      Task task_continue_1 = task_base.ContinueWith(SumEvenDiv4,
                            matrix);
35                      Task task_continue_2 = task_base.ContinueWith(FindMaxDiv3,
                            matrix);
36                      Task task_continue_3 = task_base.ContinueWith(FindMin, matrix);
37
38                      task_base.Start();
39                      Console.ReadKey();
40              }
41              static void SumEvenDiv4(Task task, object o)
42              {
43                      Console.WriteLine("Задача {0} начала выполнение...", Task.
                            CurrentId);
44                      List<List<int>> matrix = (List<List<int>>)o;
45                      int sum = 0;
46                      foreach (List<int> list in matrix)
47                      {
48                              foreach (int i in list)
49                              {
50                                      if (i % 2 == 0 && i % 4 == 0) sum += i;
51                              }
52                      }
53                      Thread.Sleep(2000);
54                      Console.WriteLine("Задача {0} завершена успешно. Сумма
                            элементов: {1}", Task.CurrentId, sum);
55              }
56              static void FindMaxDiv3(Task task, object o)
57              {
58                      Console.WriteLine("Задача {0} начала выполнение...", Task.
                            CurrentId);
59                      List<List<int>> matrix = (List<List<int>>)o;
60                      int maximum = int.MinValue;
61                      foreach (List<int> list in matrix)
62                      {
63                              foreach (int i in list)
64                              {
65                                      if (i % 3 == 0 && maximum < i) maximum = i;
66                              }
67                      }
68                      Thread.Sleep(1000);
69                      Console.WriteLine("Задача {0} завершена успешно. Максимальное
                            значение: {1}", Task.CurrentId, maximum);
70              }
71              static void FindMin(Task task, object o)
72              {
73                      Console.WriteLine("Задача {0} начала выполнение...", Task.
                            CurrentId);
```

```
                        List<List<int> > matrix = (List<List<int>>)o;
                        int minimum = int.MaxValue;
                        foreach (List<int> list in matrix)
                        {
                                foreach (int i in list)
                                {
                                        if (i < minimum) minimum = i;
                                }
                        }
                        Thread.Sleep(1500);
                        Console.WriteLine("Задача {0} завершена успешно. Минимальное
                            значение: {1}", Task.CurrentId, minimum);
                }
        }
}
```

Тестирование

Запуск программы осуществим командой, которая скомпилирует проект и запустит его в среде выполнения:

```
dotnet build; dotnet run
```

Вывод программы:

```
Основная задача 4 начала выполнение...
Основная задача 4 завершила выполнение.
Создана матрица размера 5214 на 5214
Задача 1 начала выполнение...
Задача 3 начала выполнение...
Задача 2 начала выполнение...
Задача 2 завершена успешно. Максимальное значение: 999999
Задача 3 завершена успешно. Минимальное значение: 0
Задача 1 завершена успешно. Сумма элементов: 46147132
```

Программа выполняется корректно. Задачи выполняются после выполнения главной задачи. Порядок выполнения задач продолжения не гарантирован, так как все они начинают выполняться после выполнения базовой.

Вопросы для защиты работы

1. **Что такое задача продолжения? Для чего используется данный механизм?**

   Задача продолжения - один из способов организации работы асинхронного кода, который должен дожидаться выполнения предыдущего.

2. **Сколько задач продолжения может иметь каждая задача?**

   Сколько угодно. У каждой задачи может быть несколько задач продолжения и т.д.

3. **С помощью какого метода класса Task можно задать задачу продолжения?**

   С помощью метода `ContinueWith`. Он принимает параметрами функцию, удовлетворяющую сигнатуре делегата для `Task`, и параметр `object`, который определяет передаваемые данные.

4. **Какие параметры принимает метод, представляющий задачу продолжения?**

   - Функцию, принимающую `object` и возвращающую `void`
   - Объект параметров
   - Элемент перечисления `TaskContinuationOptions`

5. **Поясните механизм определения условных задач продолжения.**

   Механизм определения условных задач продолжения определяется значением из `TaskContinuationOptions`. Например, `OnlyOnRanToCompletion` используется для того, чтобы указать, что выполнение будет только если родительская задача выполнилась успешно.

6. **Опишите методы класса TaskFactory, предназначенные для указания задач продолжения.**

   - **ContinueWhenAll(Task[], Action<Task[]>)** — создаёт задачу-продолжение, выполняемую после завершения всех указанных задач.
   - **ContinueWhenAny(Task[], Action<Task>)** — запускает задачу-продолжени после завершения любой одной задачи из набора.
   - **ContinueWhenAll<TResult>(Task<TResult>[], Action<Task<TResult>[]>, TaskContinuationOptions)** — версия для задач с результатом, с возможностью указания условий выполнения.

- **ContinueWhenAny<TResult>(Task<TResult>[], Action<Task<TResult», TaskContinuationOptions)** — выполняет продолжение после первой завершившейся задачи, возвращающей результат.

7. **Опишите механизм, использующий задачи продолжения и позволяющий запустить большое количество задач (100, 1000, …) одной командой Start.**

   - Для запуска большого количества задач используется механизм цепочек продолжений (continuation chains).
   - Первая задача создаётся и запускается методом `Start()`, а все остальные связываются с ней через `ContinueWith()`.
   - Каждая последующая задача автоматически запускается после завершения предыдущей, что позволяет организовать последовательное выполнение большого числа задач одной командой `Start()`.

Выводы

В ходе лабораторной работы были изучены принципы создания и использования задач продолжения (*continuation tasks*) в языке C#. Было показано, что механизм `ContinueWith()` позволяет организовывать последовательное и условное выполнение задач